



400-650-7088

TongRDS V2.2

命令&接口使用手册

版本：2218A01

2025 年 9 月

声明

版权

Copyright ©2025 北京东方通科技股份有限公司或其关联公司（以下简称“东方通”）。保留所有权利。

版权声明

- 本文档的所有权和知识产权归属于东方通所有。
- 除非另有明确约定，东方通不授予任何明示或暗示的许可或权利，使用者不得以任何方式将本文档中的任何内容用于商业目的。
- 东方通对本文档享有版权。本文档的所有内容，包括但不限于文字、图像、图表、图标、示意图、屏幕截图等，均受版权法和国际版权条约的保护。
- 未经东方通明确授权，任何人不得对本文档以任何形式复制、修改、传播、分发、展示或进行衍生创作。

商标声明

- **东方通**、**TongTech**标识以及其他相关东方通图形、徽标、服务名称和商标（以下统称为“东方通商标”）是东方通或其关联公司的注册商标或商标。
- 未经东方通明确授权，任何人不得以任何方式使用、复制或展示东方通商标。此外，未经东方通事先书面同意，不得将东方通商标与其他商标、标识或徽标进行混淆、链接或结合使用。
- 除非另有明确约定，本商标声明不授予任何明示或暗示的许可或权利，对东方通商标的使用须获得东方通的明确授权。
- 本文档提及的所有其他商标、标识、徽标或产品名称均为其各自所有者的财产，并可能受其各自的商标法保护。

免责声明

请在使用东方通产品之前仔细阅读本免责声明，并根据自身情况判断是否继续使用。如有任何问题，请联系我们的客户支持团队。

- 本产品的使用是基于用户自己的判断和风险评估。本文档仅作为使用指导，不对使用本产品所产生的结果做任何明示或暗示的担保。
- 东方通不对用户未按照本文档中的指导正确使用东方通产品而导致的任何损失或损害承担责任；
- 本文档可能会包含第三方提供的内容、链接或资源，这些内容由第三方自行负责。东方通对于这些内容的准确性、完整性、合法性或可靠性不承担责任。用户在使用这些内容时应自行判断和承担风险。
- 由于产品版本升级或其他原因，本文档内容会不定期更新。东方通保留在不事先通知用户的情况下随时对文档进行修改、更新或中止的权利。

如需获取有关东方通产品的许可或使用权，请联系东方通或授权代理商。任何违反本声明的行为将受到适用法律的追究。

版本变更说明

本手册的更新是累积的。因此，最新的手册版本包含对以前版本所做的所有更改。

本手册版本（2218A01）适用于 TongRDS V2.2.1.8。

文档版本	适用产品版本	更新内容	更新日期
2218A01	V2.2.1.8	更新如下内容： 更新版权声明。	2025/9/29
2217P1A01	V2.2.1.7_P1	无	2025/6/13
2217A01	V2.2.1.7	无	2025/4/18
2216P2A01	V2.2.1.6_P2	无	2024/12/6
2216P1A01	V2.2.1.6_P1	无	2024/9/29
2216A01	V2.2.1.6	无	2024/8/8
2215A01	V2.2.1.5	无	2024/6/28
2214A01	V2.2.1.4	TongRDS V2.2.1.4第一次正式发布。	2023/12/29

前言

本文档是 TongRDS V2.2 产品的用户使用手册之一，提供命令及接口的详细说明及使用方法。

阅读前注意事项

通过阅读本文档，您确认并同意自行承担因未具备必要专业背景 and 知识而导致的任何风险或后果。在使用本文档中提供的信息和指南时，请始终谨慎，并在必要时寻求专业人士建议和指导。

● 适用对象

本文档主要适用于使用本产品的系统管理员阅读，部分内容同样适用于基于本产品进行应用开发或应用部署的人员阅读。

● 专业背景

本文内容可能涉及到操作系统、服务器硬件、网络等相关领域的知识。请确保您具备相关背景 and 知识，以便更好地理解 and 应用本手册的内容。

● 技能要求

为了能够充分理解 and 应用本文档的内容，建议您具备如下技能：

1. 掌握 Linux 系统基本操作；
2. 熟悉 Java 开发语言。

● 术语和概念

本文档可能使用一些专业术语和概念。请确保您熟悉这些术语和概念，或者有能力查阅相关资料以便进一步理解。

● 实践经验

为了最大程度地受益于本文档，建议您具备一定的实践经验。这将帮助您更好地应用文档中的操作指南和建议。

请注意，本文档并不适用于没有相关专业背景 and 知识的用户。如果您对本文档的内容或所需背景有任何疑问，请在使用之前咨询相关专业人士或寻求额外的支持。

用户使用手册集

TongRDS V2.2 为您提供的用户使用手册集包含的文档有：

编号	手册集文档	说明
1	000_TongRDS_V2.2 用户手册导读	提供手册集的目录及手册版本说明。
2	001_TongRDS_V2.2 快速使用手册	提供产品介绍，安装部署产品的基本步骤及验证方式。
3	002_TongRDS_V2.2 配置参数手册	提供配置文件及参数的详细说明。
4	003_TongRDS_V2.2 管理控制台安装手册	详细介绍管理控制台的安装卸载、启动停止及

编号	手册集文档	说明
		登录。
5	004_TongRDS_V2.2 管理控制台使用手册	提供管理控制台基础信息、RDS 管理、系统监控、系统审计等功能的使用说明。
6	005_TongRDS_V2.2 客户端使用手册	介绍 TongRDS 提供的客户端程序使用说明。
7	006_TongRDS_V2.2 监控管理手册	提供安装部署监控管理组件的基本步骤。
8	007_TongRDS_V2.2 场景配置参考手册	提供常见部署模式的配置示例及验证方法。
9	008_TongRDS_V2.2 命令&接口使用手册	提供命令及接口的详细说明及使用方法。
10	009_TongRDS_V2.2 开发手册	详细介绍 TongRDS 提供的开发功能。
11	010_TongRDS_V2.2 运维手册	提供 TongRDS 常见问题的解决方案。

技术支持

东方通产品将为您提供全方位的技术支持，您可以通过以下方式获得技术支持：

网址：www.tongtech.com

电话：400-650-7088

邮箱：support@tongtech.com

您在取得技术支持时，请提供如下信息：

1. 您的姓名
2. 您的公司信息
3. 您的联系方式
4. 硬件及软件信息
5. 操作系统及其版本
6. 产品版本号
7. 日志等错误的详细信息

目录

版本变更说明.....	3
前言.....	4
目录.....	6
第 1 章 产品概述.....	9
第 2 章 命令行 Client 程序.....	10
第 3 章 命令请求.....	11
3.1 格式定义.....	11
3.1.1 命令说明.....	11
3.1.2 命令响应.....	15
3.1.3 命令用例.....	15
3.1.4 错误代码表.....	17
3.2 命令操作举例.....	17
3.2.1 设置 key= “str1” , value= “test”	17
3.2.2 查询 key= “str1” 的 value 值	18
第 4 章 特殊命令.....	19
4.1 heartbeat.....	19
4.2 check.....	19
4.3 Idle.....	19
4.4 dump.....	19
4.5 exit.....	20
第 5 章 服务监控与数据同步命令.....	21
5.1 服务器信息.....	21
5.2 客户端信息.....	21
5.3 内存信息.....	22
5.4 运行状态.....	24
5.5 集群状态.....	24
5.6 各表中存储的数据.....	24
5.7 数据同步命令.....	25
第 6 章 TongRDS 服务节点 JMX 监控接口.....	26
6.1 连接 TongRDS 的 jmx 服务.....	26
6.2 获取进程标识.....	26
6.3 获取 TongRDS 版本号.....	26
6.4 探测 redis 端口.....	26
6.5 获取进程总内存占用.....	27
6.6 获取存储数据的总内存.....	27

6.7 获取静态内存占用	27
6.8 获取动态分配的内存占用	27
6.9 获取动态分配内存的 key 的数量	27
6.10 系统可用内存总数	28
6.11 获取内存占用率	28
6.12 获取 JVM 已分配的内存总量	28
6.13 获取 JVM 空闲的内存量	28
6.14 获取 JVM 可分配的内存总量	28
6.15 取当前连接的客户端数量	29
6.16 创建的客户端总数	29
6.17 客户端连接使用率	29
6.18 TongRDS 每秒访问量.....	29
6.19 每分钟访问量	29
6.20 取节点所在集群的状态	29
6.21 取各表当前记录数	30
6.22 Jmx 客户端例程.....	30
第7章 TongRDS 哨兵节点 JMX 监控接口.....	34
7.1 连接 Sentinel 节点的 jmx 服务	34
7.2 获取进程标识	34
7.3 获取软件版本号	34
7.4 获取 JVM 已分配的内存总量	34
7.5 获取 JVM 空闲的内存量	35
7.6 获取 JVM 可分配的内存总量	35
7.7 取当前连接的客户端数量	35
7.8 Jmx 客户端例程.....	35
第8章 TongRDS 中心节点 JMX 监控接口.....	38
8.1 连接 Center 节点的 jmx 服务	38
8.2 获取进程标识	38
8.3 获取软件版本号	38
8.4 获取 license 使用量	38
8.5 获取 license 总授权量	39
8.6 获取 license 授权的过期时间	39
8.7 获取 JVM 已分配的内存总量	39
8.8 获取 JVM 空闲的内存量	39
8.9 获取 JVM 可分配的内存总量	39
8.10 获取当前连接的客户端数量	40
8.11 Jmx 客户端例程.....	40

附录 A 客户端 API 演示程序.....	43
附录 B 术语说明	47

第1章 产品概述

随着行业竞争的日趋激烈，业务的实时性和可用性要求越来越高，数据量也越来越大。同时随着 IT 技术的不断发展，系统能力的不断提升，也为这种要求实现提供了可能。但是，IT 能力和业务需求之间的鸿沟是永远存在的，IT 能力永远会落后于业务需求。为了解决行业应用对大数据量、高并发实时访问、高可靠的业务支持的需求，TongRDS 产品应运而生。

TongRDS 产品是基于我方对于高性能与高可靠业务的充分理解，结合我方在内存数据库技术方面的多年研究开发经验，推出的新一代共享内存数据库产品。

本手册详细介绍 TongRDS 的 RDSCP 协议命令和接口的使用方法。

第2章 命令行Client程序

TongRDS 提供命令行连接服务的 Client 程序，可用于验证 TongRDS 启动是否正常以及日常运行维护。详细介绍请参考《005_TongRDS_V2.2 客户端使用手册》中“RDSCP 协议命令”章节。

第3章 命令请求

TongRDS 采用标准 TCP/IP 方式通信，命令请求与应答采用行方式完成，即每个命令请求为 1 行，以换行符结束；应答也采用同样的行方式定义。（测试可采用 telnet 方式直接连接服务端口）。

3.1 格式定义

命令请求格式定义如下：

sess_id command table_id key value <key1 key2 ... keyn>

其中：

- **sess_id**: 流水号，必选项，该项只用于客户端校验服务器程序。服务程序收到流水号后不做其他处理，直接返回给客户端。如果客户端不需要校验，可以设置成固定值。
- **command**: 命令，必选项。目前支持客户端调用的命令有 set（写数据）、get（按主键取 value 数据）、get<n>（按非唯一索引 keyn 取 value 数据，程序支持 3 个索引的联合查询，如果有多条会同时返回，各 value 间用空格分割）（例如：'1 get1 ssslaab'）、get<n>key<m>（按非唯一索引 keyn 取对应记录的非唯一索引 keym 的值，m=0 或缺省 m 时，取对应记录唯一索引 key 的值，如果有多条会同时返回，各返回值间用空格分隔）（例如：'1 get1key 1 ssslaab'）、del（删除指定的数据）。
- **table_id**: 需要操作的数据表的编号，与配置文件中的编号对应
- **key**: 主键，必选项。value 数据的唯一索引。字符串，最大长度支持 128bytes
- **value**: 值，可选项。字符串，最大长度为 512bytes
- **key1**: value 的第一不唯一索引，可选项。
-
- **keyn**: value 的第 n 个不唯一索引，可选项。

3.1.1 命令说明

本软件提供外部命令行方式的接口操作。操作命令和命令格式、返回信息等介绍如下。

3.1.1.1 get 命令

根据输入条件取数据。输入条件可以是主键或索引，做联合索引查询时最多可以关联 3 个索引查询，返回的数据可以是 value 字段，也可以是主键或索引字段。

- 通过主键取 value: “get 主键值”。
- 通过索引 1 取所有匹配记录的 value: “get1 索引 1 值”。
- 通过索引 1、2 联合匹配取 value: “get1&2 索引 1 值 索引 2 值”。

- 通过索引 1 取所有匹配记录的索引 2: “get1key2 索引 1 值”。

举例, 根据索引 1 和索引 2 查表 1 的 value 命令如下:

“1 get1&2 1 123 456”

3.1.1.2 set 命令

向系统内写数据。如果系统内无对应主键的数据则增加。如果已有对应的数据, 要看 updateModel 配置, 如果配置为 insert 则返回 28 号错误插入失败; 否则更新现有记录。

3.1.1.3 upd 命令

向系统内更新数据, 和 set 的区别是 upd 命令要求系统内已有对应的未过期的数据, 否则会报 1 号错误更新失败。

upd 命令和 set 命令的区别是: 1、upd 命令要求系统内必须有未过期的对应的数据; 2、upd 命令与 updateModel 配置无关。

3.1.1.4 count 命令

根据输入的索引条件进行统计, 并返回统计结果。输入条件只能是索引, 可以最多 3 个索引做联合查询, 命令格式如下:

- 根据索引 1 统计记录数: count1 索引 1 值。
- 根据索引 1、2 统计记录数: count1&2 索引 1 值 索引 2 值。

3.1.1.5 ttl (pttl) 命令

根据输入的 key 值查询指定记录的到期时间, ttl 返回距离到期时刻的秒数, pttl 返回举例到期时刻的毫秒数。返回-1 说明记录永不过期; 返回-2 是记录没找到或已经过期。

3.1.1.6 del 命令

根据输入条件删除记录。输入条件可以是主键, 也可以是索引。命令格式如下:

- 根据主键删除记录: del 主键。
- 根据索引 1 删除记录: del1 索引 1 值。
- 根据索引 1、2 删除记录: del1&2 索引 1 值 索引 2 值。

3.1.1.7 dump 命令

根据指定的索引条件输出 value 数据:

根据索引 1 查数据：dump1 索引 1 值。

3.1.1.8 incr 命令

根据输入的 key 值，对应的 value 进行增值操作。支持 value 的类型为 bytes、Int、Long 等，可成功执行的前提是 key 存在。可制定增加的数值，缺省是加 1。命令举例：

```
1 incr 1 123 2
```

3.1.1.9 decr 命令

根据输入的 key 值，对应的 value 进行减值操作。支持 value 的类型为 bytes、Int、Long 等，可成功执行的前提是 key 存在。可制定增加的数值，缺省是减 1。

3.1.1.10 list 命令组

实现类似 Redis 的 list 数据类型，程序增加 Variable 类型支持该功能。Variable 类型在 byte 数组的基础上增加 list 的支持。

当 list 中的总项数超过预定义数值（64 个）时，或者总长度超过 value 预设长度时，系统会创建一个 Arraylist 对象专门保存该项的数据。

list 命令组包含如下命令：

- **lpush**：在原字符串左边叠加（key 不存在时会新增）。如原来的 value 是“456”，执行 1 lpush 1 key 123 以后 value 变成“123\t456”。命令举例：

```
20 lpush 1 list1 two
```

```
21 lpush 1 list1 one
```

- **rpush**：在原字符串右边叠加（key 不存在时会新增）。命令举例：

```
22 rpush 1 list1 there
```

- **lpop**：删除原字符串左起第一项，并返回被去除项的内容。如果 value 中只剩最后 1 项，返回该项，并删除 key。下次同样的操作会返回“数据未找到”的错误。命令举例：

```
23 lpop 1 list1
```

- **rpop**：删除原字符串右起第一项，并返回被去除的项的内容。处理流程类似 lpop。命令举例：

```
24 rpop 1 list1
```

- **ltrim**：只保留原字符串左起的指定项的文本，后面的文本会被删除，如果总项数小于输入值则 value 值无变化。例如，只保留列表左边 5 项：

```
25 ltrim 1 list1 5
```

- **rtrim**：只保留原字符串右起的指定项的文本。操作结果类似于 ltrim。

- **lfetch:** 取 list 的值。命令举例:

```
26 lfetch 1 list1
```

- **lcount:** 计算指定 key 对应的 value 的 list 个数。命令举例:

```
27 lcount 1 list1
```

3.1.1.11 set 命令组

实现类似 Redis 的 set 数据类型, 和 list 的区别: 1、集合中的各项不能重复 (去重); 2、无顺序。程序增加 Variable 类型支持该功能, Variable 类型在 byte 数组的基础上增加 set 的支持。

当 set 中的总项数超过预定义数值 (64 个) 时, 或者总长度超过 value 预设长度时, 系统会创建一个 hashset 对象专门保存该项的数据。

- **sadd:** 增加一项。举例:

```
31 sadd 1 s1 java
```

```
31 sadd 1 s1 go
```

```
31 sadd 1 s1 python
```

```
31 sadd 1 s1 c++
```

- **srem:** 删除指定项。举例: 32 srem s1 go
- **spop:** 随机删除 1 项, 并返回删除的内容。举例: 33 spop 1 s1
- **smembers:** 返回 set 中的所有项。举例: 34 smembers 1 s1
- **sismember:** 判断输入项是否在 set 中, 在则返回 1。
- **scard:** 返回当前 set 中的总项数。举例: 35 scard 1 s1

3.1.1.12 hash 命令组

实现类似 Redis 的 hash 数据类型, 程序增加 Variable 类型支持该功能。Variable 类型在 byte 数组的基础上增加 hash 的支持, 将字符 (\r) 作为字段的分隔符, 将 hash 内容转换为 “key ‘\r’ value” 的形式然后各项拼接到一起保存到 value 中。换言之, value 还是整段的 bytes 字符串, 当 hash 命令组操作时, 会按字符 \r 将整段字符串分解成各 key、value 项处理。

当 hash 中的总项数超过预定义数值 (64 个) 时, 或者总长度超过 value 预设长度时, 系统会创建一个 hashMap 对象专门保存该项的数据。

- **hset:** 设置 1 项。举例:

```
41 hset 1 h1 age 20
```

```
41 hset 1 h1 name jobs
```

- **hincrby**: 增加本集合内指定项的值（整数）。举例：42 hincrby 1 h1 age 10
- **hdel**: 删除本集合内的指定项。
- **hget**: 返回本集合中指定项的内容。
- **hgetall**: 返回本集合的全部内容，返回格式为 key1、value1、key2、value2、、、。举例：43 hgetall 1 h1
- **hlen**: 返回本集合中的总项数。
- **hkeys**: 返回本集合中的全部 key 值，返回格式为：key1、key2、key3、、、。
- **hvals**: 返回本集合中全部 value 值，返回格式同 hkeys 命令。
- **hexists**: 判断指定的 key 是否在本集合中。

3.1.2 命令响应

请求相应格式定义如下：

response_id status < error_code error_msg> value <values...>

- **response_id**: 响应 ID，对应命令请求中的 sess_id，必填项。
- **status**: 命令执行结果状态：“ok”为成功，“error”为失败，必填项。
- **error_code**: 当 status 为“error”时，返回错误编码。编码定义见[错误代码表](#)。
- **value**: 查询命令成功时返回 value 数据值。可选项，查询命令的返回内容。
- **<values...>**: 查询多项结果时返回其他的 value 值，个项用空格分隔。可选项。

3.1.3 命令用例

命令用例如下：

```
# upd 命令更新需要先有数据
1 upd 1 aaa bbb 111 222 333
1 error 29 error in upd

# set 写数据
1 set 1 aaa ccc 112 223 334
1 ok

# get 读数据
1 get 1 aaa
```

```

1 ok ccc

# upd 更新数据

1 upd 1 aaa bbb 111 222 333

1 ok

1 get 1 aaa

1 ok bbb

# 写数据

1 set 1 bbb ccc 111 333 333

1 ok

1 set 1 ccc ddd 111 444 555

1 ok

1 set 1 eee fff 222 333 333

1 ok

# 根据索引 1 查询

1 get1 1 111

1 ok bbb ccc ddd

# 根据索引 3 查询

1 get3 1 333

1 ok bbb ccc fff

# 根据索引 1、索引 3 联合查询

1 get1&3 1 111 333

1 ok bbb ccc

# dump 根据索引 1 取 value 数据

1 dump1 1 111

bbb

ccc

ddd

dump ok

# dump 根据索引 3 取 value 数据

1 dump3 1 333

bbb

ccc

fff

```


dump ok

3.1.4 错误代码表

错误代码	错误原因
0	操作成功
-1	请求格式非法
-2	请求中的 command 参数为空
-3	请求中的 key 参数为空
-4	不能解析请求的命令
-5	table_id 参数非法
-6	set、upd 命令时 value 为空
-7	set、upd 命令时索引为空
-9	异常错
-10	系统未启动完成，不接收同步以外的命令
1	未找到需要查询的数据
2	写/更新数据（set）错误
21	key 值超长
22	value 值超长
23	某个 index 超长，具体需要看 server 的日志
26	写数据时加锁失败
27	服务器数据更新方式配成 insert 方式（Server.Table<n>.updateModel = insert），且输入数据的 key 在系统中已存在
28	服务器自动覆盖方式配成 false，不自动覆盖较旧的数据（Server.AutoOverlay = false），且内存表已满
3	删除数据（del）错误
4	匹配到的数据过多
41	同步数据比内存表中的数据旧，不更新
42	同步的数据已经被删除，不允许插入

3.2 命令操作举例

3.2.1 设置key=“str1”，value=“test”

命令：

```
10 set 1 str1 test
```

其中：

- **10：**是 session-id，帮助客户端校验请求和响应的对应关系（通常是客户端自己创建一个流水号）；

- **set:** 是设置字符串的命令;
- **1:** 是第一张表 (对应配置文件的 table1);
- **str1:** 是 key 的值;
- **test:** 是 value 的值, 各项用空格分隔, 整条命令用\n 结束。

响应

10 ok

其中: **10** 是客户端发起的流水号; **ok** 表示设置成功。

3.2.2 查询key=“str1”的value值

命令:

11 get 1 str1

响应:

11 ok test

第4章 特殊命令

特殊命令为简单命令格式，无序列号，也没有其它参数。

4.1 heartbeat

心跳（heartbeat）命令可用于客户端检测服务器是否可用，如果服务器已经启动完成并可提供服务，收到该命令时将返回“hb”+服务器启动完成时的时间；否则服务器会断开当前连接。用例如下：

```
test[mysql:/data1/mysql]=> telnet 0 16200
Trying 0.0.0.0...
Connected to 0.
Escape character is '^]'.
heartbeat
hb 1400559146481
```

4.2 check

check 命令用于内部的数据同步线程检测服务器是否能正常监听端口，服务器收到该命令后返回“ck”+服务器开始启动的时间+服务器唯一标志（唯一标志格式为“16 进制 ID 号-端口号”，同步线程用该标志判断当前连接的对端是否是自己，如果是自己则该连接不需要同步数据）。该命令只有同步包中的 Connector 类使用。用例如下：

```
test[mysql:/data1/mysql/memdb/bin]=> telnet 0 16200
Trying 0.0.0.0...
Connected to 0.
Escape character is '^]'.
check
ck 1467963869641 79a80003ba2c8aa6-16200
```

4.3 Idle

idle 命令只用于保持连接。如果客户端长时间没有传输数据，可以用该命令定期检测连接状态，并保持连接。服务器收到该命令后返回“ok”。

4.4 dump

dump 命令用于持久化当前内存中的数据。程序会在主目录的 var 子目录下创建文件名为 dump-{table-id}.dat 的文件组，对应内存中的表。文件采用 BerkeleyDB 格式存储，该文件以 key-value 对的形式保存数

据。

4.5 exit

退出（exit）命令用于终止当前连接，服务器收到该命令后会立即关闭当前连接。用例如下：

```
test[mysql:/data1/mysql]=> telnet 0 16200
Trying 0.0.0.0...
Connected to 0.
Escape character is '^]'.
exit
Connection to 0 closed by foreign host.
test[mysql:/data1/mysql]=>
```

第5章 服务监控与数据同步命令

TongRDS 通过仿真的 Redis 命令 `info` 可以查询到相关运行信息，通过 `rdsync` 命令可以从指定节点同步数据到本节点。

5.1 服务器信息

登录 Redis 仿真端口，执行 `info server` 命令，返回如下信息：

```
TongRDS_Version:2.2.1.1_P4 (build 21-12-10 10:17:21)
redis_version:3.2.13
redis_mode:standalone
run_id:9098185680aae529-6200
tcp_port:6379
uptime_in_seconds:1215
uptime_in_days:0
```

其中：

- **TongRDS_Version:** TongRDS 版本信息和软件编译时间，可用于精确区分不同版本。
- **redis_version:** 仿真 redis 的版本号。
- **redis_mode:** TongRDS 运行模式，如果是集群状态，显示 `cluster`，否则显示 `standalone`。
- **os:** 运行 TongRDS 的操作系统=。
- **tcp_port:** 仿真 Redis 的端口号。
- **process_id:** TongRDS 进程 id。

5.2 客户端信息

执行 `info clients` 获取客户端信息如下：

```
# Clients
connected_clients:1
max_clients:200
connected_ratio:0.005
127.0.0.1-connected:1
127.0.0.1-received_in_minute:0
```

其中：

- **connected_clients:** 当前总的客户端连接数。
- **max_clients:** 可同时接入的最大连接数。
- **connected-ratio:** 当前已有连接数和最大连接数之间的比值。
- **IP-connected:** 指定 ip 的客户端的总连接数。
- **IP-received_in_minute:** 指定 ip 的客户端最近 1 分钟创建连接的总数量。

5.3 内存信息

执行 **info memory** 获取内存统计信息如下：

```
# Memory
used_memory:3080000
used_memory_human:2.93M
used_memory_peak:3080000
used_memory_peak_human:2.93M
used_static_memory:3080000
used_static_memory_human:2.93M
used_dynamic_size:0
used_dynamic_memory:0
used_dynamic_memory_human:0
used_dynamic_memory_peak:0
used_dynamic_memory_peak_human:0
used_memory_rss:126877696
used_memory_rss_human:121M
used_memory_ratio:0.02
used_memory_lua:0
jvm_allocated_memory:126877696
jvm_allocated_memory_human:121M
jvm_uesed_memory:20666128
jvm_uesed_memory_human:19.7M
jvm_free_memory:106211568
jvm_free_memory_human:101.29M
jvm_max_memory:1871708160
jvm_max_memory_human:1.74G
```

```
total_system_memory:8421134336
total_system_memory_human:7.84G
```

其中：

- **used_memory:** 存储数据占用的总内存。
- **used_memory_human:** 易读方式显示存储数据占用的总内存。
- **used_memory_peak:** 启动以来存储数据占用的总内存的峰值。
- **used_memory_peak_human:** 易读方式显示启动以来存储数据占用的总内存的峰值。
- **used_static_memory:** 存储数据占用静态分配的内存。
- **used_static_memory_human:** 易读方式显示存储数据占用静态分配的内存。
- **used_dynamic_size:** 系统内超大单条数据的总条数。
- **used_dynamic_memory:** 存储超大单条数据申请的动态内存总量。
- **used_dynamic_memory_human:** 易读方式显示存储超大单条数据申请的动态内存总量。
- **used_dynamic_memory_peak:** 系统启动以来存储超大单条数据申请的动态内存总量的峰值。
- **used_dynamic_memory_peak_human:** 易读方式显示系统启动以来存储超大单条数据申请的动态内存总量的峰值。
- **used_memory_rss:** 系统进程占用的总内存。
- **used_memory_rss_human:** 易读方式显示系统进程占用的总内存。
- **used_memory_ratio:** 占用内存的使用率 (used_memory/used_memory_rss)，通常小于 1。
- **used_memory_lua:** 缓存的 lua 脚本占用内存估算。
- **jvm_allocated_memory:** jvm 分配的总内存大小。
- **jvm_allocated_memory_human:** 易读方式显示 jvm 分配的总内存大小。
- **jvm_uesed_memory:** jvm 已经使用的内存大小。
- **jvm_uesed_memory_human:** 易读方式显示 jvm 已经使用的内存大小。
- **jvm_free_memory:** jvm 中空闲内存大小。
- **jvm_free_memory_human:** 易读方式显示 jvm 中空闲内存大小。
- **jvm_max_memory:** jvm 可分配的最大内存。
- **jvm_max_memory_human:** 易读方式显示 jvm 可分配的最大内存。
- **total_system_memory:** 系统总物理内存。
- **total_system_memory_human:** 易读方式显示系统总物理内存。

5.4 运行状态

执行 **info stats** 获取 TongRDS 运行状态指标：

```
# Status
total_connections_received:1
total_connections_ratio:1.0
total_commands_processed:-1
process_speed:0
process_speed_minute:3.0
```

其中：

- **total_connections_received**：系统启动以来创建的客户端连接总数。
- **total_connections_ratio**：创建连接的利用率（不大于 1）。
- **process_speed**：当前每秒处理的请求量。
- **process_speed_minute**：最近 1 分钟处理的请求总量。

5.5 集群状态

执行 **info cluster** 命令返回 TongRDS 集群状态。

```
# Cluster
cluster_enabled:0
```

其中：

cluster_enabled：服务是否运行在集群状态。

如果 cluster_enabled 为 1，说明运行在集群状态。可以进一步运行 **cluster nodes** 命令查看集群包含的各节点的状态。

5.6 各表中存储的数据

执行 **info keyspace** 命令查询当前服务配置的各表（每个表占 1 行）中存储的数据量。

```
# Keyspace
db0:keys=30,expires=0,avg_ttl=0
```

上述结果显示当前服务只配置了一个表（Table1），table1 中当前共有 30 条记录。其他项数据是为了模拟 redis 的数据结构，无实际意义。

5.7 数据同步命令

执行 **rdsync host port [any]** 命令从 host: port 指定的节点向本节点同步数据，可选参数 “any” 指定是否不要求同步节点必须在 dynamic.xml 的同步列表内，缺省时输入的地址必须在同步列表内，加入 any 参数则不验证此项，允许从任意 rds 节点同步数据。

注意：此处的 port 是 RDS 协议端口号（缺省为 6200），非仿真 Redis 协议的端口号。

+OK

上述结果显示 **rdsync** 命令执行成功，如果数据同步失败则返回数字 “0”，如果出现异常错误则会返回错误信息。

第6章 TongRDS服务节点JMX监控接口

JMX 是在 TongRDS 中用于获取监视量的工具，本节将介绍在 TongRDS 中使用 JMX 获取监视量的方法以及 JMX 对象 ObjectName 接口的对应表。

6.1 连接TongRDS的jmx服务

客户端采用如下代码连接本机的 rds 服务端，并创建 MBean 的 ObjectName 对象：

```
JMXServiceURL url = new JMXServiceURL("service:jmx:rmi:///jndi/rmi://localhost:29094/jmxrmi");

JMXConnector jmxc = JMXConnectorFactory.connect(url, null);
MBeanServerConnection mbsc = jmxc.getMBeanServerConnection();

//ObjectName 的名称与前面注册时候的保持一致
ObjectName mbeanName = new ObjectName("jmxBean:name=rdsCore");
```

6.2 获取进程标识

执行如下代码获取当前 TongRDS 的进程标识：

```
// 实例标识名
String id = (String) mbsc.getAttribute(mbeanName, "Id");
```

6.3 获取TongRDS版本号

执行如下代码获取当前 TongRDS 服务的版本信息：

```
// 版本号
String version = (String) mbsc.getAttribute(mbeanName, "Version");
```

Version 为字符串形式返回的版本信息。

6.4 探测redis端口

执行如下代码探测 redis 仿真端口是否正常：

```
boolean isOk = (boolean) mbsc.getAttribute(mbeanName, "Ping");
```

jmx 服务器会模拟客户端创建 socket 连接到 redis 端口，并通过 **PING-PONG** 命令探测端口状态，返回 true 说明端口正常。

6.5 获取进程总内存占用

执行如下代码获取 TongRDS 进程占用的总内存：

```
// jvm 占用的总内存
Long memoryTotal = (Long) mbsc.getAttribute(mbeanName, "MemoryTotal");
```

6.6 获取存储数据的总内存

存储数据的总内存是用于存储有用数据而占用的总的内存，包括静态内存占用和动态内存占用两部分。获取数据总内存的代码如下：

```
// 储存数据占用的内存
Long usedMemory = (Long) mbsc.getAttribute(mbeanName, "UsedMemory");
```

6.7 获取静态内存占用

执行如下代码获取 TongRDS 中静态分配的内存大小：

```
// cfg 配置文件中配置的内存大小
Long memoryStatic = (Long) mbsc.getAttribute(mbeanName, "MemoryStatic");
```

6.8 获取动态分配的内存占用

执行如下代码获取 TongRDS 中为大对象分配的总内存数：

```
// 动态分配的超长数据的总内存大小
Long memoryDynamic = (Long) mbsc.getAttribute(mbeanName, "MemoryDynamic");
```

6.9 获取动态分配内存的key的数量

执行如下代码获取 TongRDS 中需要动态分配内存的 key 的总数：

```
// 动态分配的超长数据的总个数
Long sizeDynamic = (Long) mbsc.getAttribute(mbeanName, "SizeDynamic");
```

6.10 系统可用内存总数

执行如下代码获取 TongRDS 运行环境的可用内存总数：

```
// 系统可用内存
Long memoryTotalAvailable = (Long) mbsc.getAttribute(mbeanName, "TotalPhysicalMemory");
```

6.11 获取内存占用率

内存占用率为存储数据的总内存/进程占用的总内存。该指标用于描述进程占用的系统内存的使用效率。执行如下代码获取进程的内存占用率：

```
// 内存使用率
Double memoryUsedRatio = (Double) mbsc.getAttribute(mbeanName, "MemoryUsedRatio");
```

6.12 获取JVM已分配的内存总量

该值表示 JVM 已经从操作系统中申请的内存空间。执行如下代码获取 JVM 已分配的内存：

```
// jvm 已经分配的内存
Long jvmAllocated = (Long) mbsc.getAttribute(mbeanName, "JvmAllocated");
```

6.13 获取JVM空闲的内存量

该值表示 JVM 已经从操作系统中申请的内存空间中尚未使用的部分。执行如下代码获取 JVM 已分配的内存：

```
// jvm 已经分配但空闲的内存
Long jvmFree = (Long) mbsc.getAttribute(mbeanName, "JvmFree");
```

注：JVM 已使用的内存总量可使用上述 2 项的值进行减法计算获得。

6.14 获取JVM可分配的内存总量

该值表示当前进程可分配的内存总量（可由启动参数-Xmx 指定）执行如下代码获取 JVM 已分配的内存：

```
// jvm 可分配的最大内存
Long jvmMax = (Long) mbsc.getAttribute(mbeanName, "JvmMax");
```

6.15 取当前连接的客户端数量

执行如下代码获取 TongRDS 当前正在连接的客户端总数：

```
// 当前接入的 client 连接数量
Long clientCurrentConnections = (Long) mbsc.getAttribute(mbeanName, "ClientCurrentConnections");
```

6.16 创建的客户端总数

执行如下代码获取 TongRDS 自从启动以来创建的 client 连接总数：

```
// 自启动以来创建过的客户端连接总数
Long clientTotalConnections = (Long) mbsc.getAttribute(mbeanName, "ClientTotalConnections");
```

6.17 客户端连接使用率

执行如下代码获取 TongRDS 创建 client 连接的效率：

```
// 客户端连接使用率
Double connectionsRatio = (Double) mbsc.getAttribute(mbeanName, "ConnectionsRatio");
```

6.18 TongRDS每秒访问量

执行如下代码获取 TongRDS 当前时刻每秒的访问量：

```
// 当前每秒钟处理请求量
Long processSecond = (Long) mbsc.getAttribute(mbeanName, "ProcessSecond");
```

6.19 每分钟访问量

执行如下代码获取 TongRDS 最近一分钟内的访问量：

```
// 当前每分钟处理请求量
Long processMinute = (Long) mbsc.getAttribute(mbeanName, "ProcessMinute");
```

6.20 取节点所在集群的状态

执行如下代码返回字符串数组，描述当前集群的状态：

```
// 集群状态，list 中的每 1 项（String 数组）代表一个 cluster 的分片

// String 数组为定长，各项内容分别为：分片主节点名称、主节点地址、主节点状态、分片状态
//（success 或 fail）、分配槽位号列表

List<String[]> clusters = (List<String[]>) mbsc.getAttribute(mbeanName, "ClusterStatus");
```

该方法返回集群状态，返回值是一个由字符串数组组成的 list，每个集群分片占一行，每行中的字符串数据依次为：主节点名、主节点地址、主节点状态、分片状态、分片槽位号。

如果当前节点未工作在集群状态，则返回 null。

6.21 取各表当前记录数

执行如下代码获取 TongRDS 中各表的记录总数：

```
// 各表中数据量

Map<String, Integer> keys = (Map<String, Integer>) mbsc.getAttribute(mbeanName, "Keyspace");
```

6.22 Jmx客户端例程

以下为 jmx 演示例程：

```
import javax.management.*;
import javax.management.remote.JMXConnector;
import javax.management.remote.JMXConnectorFactory;
import javax.management.remote.JMXServiceURL;
import java.util.List;
import java.util.Map;

public class JmxTest {
    public static void main(String[] args) throws Exception {
        JMXServiceURL url = new JMXServiceURL("service:jmx:rmi:///jndi/rmi://localhost:29094/jmxrmi");

        JMXConnector jmx = JMXConnectorFactory.connect(url, null);
        MBeanServerConnection mbsc = jmx.getMBeanServerConnection();

        //ObjectName 的名称与前面注册时候的保持一致
        ObjectName mbeanName = new ObjectName("jmxBean:name=rdsCore");
```

```
// 版本号
String version = (String) mbsc.getAttribute(mbeanName, "Version");

boolean isOk = (boolean) mbsc.getAttribute(mbeanName, "Ping");

// jvm 占用的总内存
Long memoryTotal = (Long) mbsc.getAttribute(mbeanName, "MemoryTotal");

// 储存数据占用的内存
Long usedMemory = (Long) mbsc.getAttribute(mbeanName, "UsedMemory");

// cfg 配置文件中配置的内存大小
Long memoryStatic = (Long) mbsc.getAttribute(mbeanName, "MemoryStatic");

// 动态分配的超长数据的总内存大小
Long memoryDynamic = (Long) mbsc.getAttribute(mbeanName, "MemoryDynamic");

// 动态分配的超长数据的总个数
Long sizeDynamic = (Long) mbsc.getAttribute(mbeanName, "SizeDynamic");

// 系统可用内存
Long memoryTotalAvailable = (Long) mbsc.getAttribute(mbeanName, "TotalPhysicalMemory");

// 当前接入的 client 连接数量
Long clientCurrentConnections = (Long) mbsc.getAttribute(mbeanName, "ClientCurrentConnections");

// 自启动以来创建过的客户端连接总数
Long clientTotalConnections = (Long) mbsc.getAttribute(mbeanName, "ClientTotalConnections");

// 客户端连接使用率
Double connectionsRatio = (Double) mbsc.getAttribute(mbeanName, "ConnectionsRatio");
```

```
// 当前每秒钟处理请求量
Long processSecond = (Long) mbsc.getAttribute(mbeanName, "ProcessSecond");

// 最近一分钟处理请求量
Long processMinute = (Long) mbsc.getAttribute(mbeanName, "ProcessMinute");

// 集群状态，list 中的每 1 项（String 数组）代表一个 cluster 的分片
// String 数组为定长，各项内容分别为：分片主节点名称、主节点地址、主节点状态、分片状态
// (success 或 fail)、分配槽位号列表
List<String[]> clusters = (List<String[]>) mbsc.getAttribute(mbeanName, "ClusterStatus");

// 各表中数据量
Map<String, Integer> keys = (Map<String, Integer>) mbsc.getAttribute(mbeanName, "Keyspace");

System.out.println("Version = " + version);
System.out.println("Ping: " + isOk);
System.out.println("memoryStatic = " + memoryStatic);
System.out.println("memoryDynamic = " + memoryDynamic);
System.out.println("sizeDynamic = " + sizeDynamic);
System.out.println("memoryTotalAvailable = " + memoryTotalAvailable);
System.out.println("clientCurrentConnections = " + clientCurrentConnections);
System.out.println("clientTotalConnections = " + clientTotalConnections);
System.out.println("processSecond = " + processSecond);
System.out.println("processMinute = " + processMinute);

if (keys != null) {
    for (String name : keys.keySet()) {
        System.out.println(name + ": " + keys.get(name));
    }
}

if (clusters != null) {
    System.out.println("Cluster enabled:");
    for (int i = 0; i < clusters.size(); i++) {
```



```

        System.out.print(" Shard-" + i + ": ");
        String[] cluster = clusters.get(i);
        System.out.println(String.format(" %22s %22s %13s %7s %s"
            , cluster[0], cluster[1], cluster[2], cluster[3], cluster[4]));
    }
} else {
    System.out.println("Cluster support disabled");
}
}
}

```

第7章 TongRDS哨兵节点JMX监控接口

本节介绍采用 jmx 接口监视哨兵（Sentinel）节点运行时状态信息。

7.1 连接Sentinel节点的jmx服务

客户端采用如下代码连接本机的 Sentinel 服务端，并创建 MBean 的 ObjectName 对象：

```
JMXServiceURL url = new JMXServiceURL("service:jmx:rmi:///jndi/rmi://localhost:29096/jmxrmi");

JMXConnector jmxc = JMXConnectorFactory.connect(url, null);
MBeanServerConnection mbsc = jmxc.getMBeanServerConnection();

//ObjectName 的名称与前面注册时候的保持一致
ObjectName mbeanName = new ObjectName("jmxBean:name=rdsSentinel");
```

7.2 获取进程标识

执行如下代码获取当前 TongRDS 的进程标识：

```
// 实例标识名
String id = (String) mbsc.getAttribute(mbeanName, "Id");
```

7.3 获取软件版本号

执行如下代码获取当前 TongRDS 服务的版本信息：

```
// 版本号
String version = (String) mbsc.getAttribute(mbeanName, "Version");
```

Version 为字符串形式返回的版本信息。

7.4 获取JVM已分配的内存总量

该值表示 JVM 已经从操作系统中申请的内存空间。执行如下代码获取 JVM 已分配的内存：

```
// jvm 已经分配的内存
Long jvmAllocated = (Long) mbsc.getAttribute(mbeanName, "JvmAllocated");
```

7.5 获取JVM空闲的内存量

该值表示 JVM 已经从操作系统中申请的内存空间中尚未使用的部分。执行如下代码获取 JVM 已分配的内存：

```
// jvm 已经分配但空闲的内存
Long jvmFree = (Long) mbsc.getAttribute(mbeanName, "JvmFree");
```

注：JVM 已使用的内存总量可使用上述 2 项的值进行减法计算获得。

7.6 获取JVM可分配的内存总量

该值表示当前进程可分配的内存总量（可由启动参数-Xmx 指定）执行如下代码获取 JVM 已分配的内存：

```
// jvm 可分配的最大内存
Long jvmMax = (Long) mbsc.getAttribute(mbeanName, "JvmMax");
```

7.7 取当前连接的客户端数量

执行如下代码获取 TongRDS 当前正在连接的客户端总数：

```
// 当前接入的 client 连接数量
Long clientCurrentConnections = (Long) mbsc.getAttribute(mbeanName, "ClientCurrentConnections");
```

7.8 Jmx客户端例程

以下为 jmx 演示例程：

```
public void getJmx(Statistics statistics/* pojo */) {
    if (jmxUrl != null && !jmxUrl.startsWith("service:jmx")) {
        jmxUrl = "service:jmx:rmi:///jndi/rmi://" + jmxUrl + "/jmxrmi";
    }

    JMXConnector jmxc = null;
    try {
        JMXServiceURL url = new JMXServiceURL(jmxUrl);
```

```

jmx = JMXConnectorFactory.connect(url, null);

// 程序能运行到这里说明已经连接上 jmx-server 了
statistics.setAlive(true);

MBeanServerConnection mbsc = jmx.getMBeanServerConnection();

//ObjectName 的名称与前面注册时候的保持一致
ObjectName mbeanName = new ObjectName("jmxBean:name=rdsSentinel");

// 实例标识名
statistics.setJd((String) mbsc.getAttribute(mbeanName, "Id"));

// 版本号
statistics.setVersion((String) mbsc.getAttribute(mbeanName, "Version"));

// jvm 已经分配的内存
statistics.setJvmAllocated((Long) mbsc.getAttribute(mbeanName, "JvmAllocated"));

// jvm 已经分配但空闲的内存
statistics.setJvmFree((Long) mbsc.getAttribute(mbeanName, "JvmFree"));

// jvm 可分配的最大内存
statistics.setJvmMax((Long) mbsc.getAttribute(mbeanName, "JvmMax"));

// 当前接入的 client 连接数量
statistics.setCurrentConnections((Long) mbsc.getAttribute(mbeanName, "ClientCurrentConnections
"));

LOGGER.infoLog("SentinelDeployer::check() {} read jmx-server '{}' ok.", name, jmxUrl);
} catch (AttributeNotFoundException anf) {
    if (LOGGER.isDebugEnabled()) {
        LOGGER.errorLog(anf, "SentinelDeployer::check() {} read jmx-server '{}' failed: {}", name, jmx
Url, anf.getMessage());
    }
}

```

```

    } else {
        LOGGER.errorLog("SentinelDeployer::check() {} read jmx-server '{}' failed: {}", name, jmxUrl,
anf.getMessage());
    }
} catch (IOException ioe) {
    this.statistics.setAlive(false);
    if (LOGGER.isDebugEnabled()) {
        LOGGER.infoLog(ioe, "SentinelDeployer::check() {} connect to jmx-server '{}' failed: {}", nam
e, jmxUrl, ioe);
    } else {
        LOGGER.infoLog("SentinelDeployer::check() {} connect to jmx-server '{}' failed: {}", name, jm
xUrl, ioe);
    }
} catch (Throwable t) {
    if (LOGGER.isDebugEnabled()) {
        LOGGER.infoLog(t, "SentinelDeployer::check() {} jmx failed from '{}' failed: {}", name, jmxUr
l, t);
    } else {
        LOGGER.infoLog("SentinelDeployer::check() {} jmx failed from '{}' failed: {}", name, jmxUrl,
t);
    }
} finally {
    try {
        jmx.close();
    } catch (Throwable e) {
//        e.printStackTrace();
    }
}

    LOGGER.debugLog("SentinelDeployer::check() {}'s statistics: {}", this.name, this.statistics);
}

```

第8章 TongRDS中心节点JMX监控接口

本节介绍采用 jmx 接口监视中心（Center）节点运行时状态信息。

8.1 连接Center节点的jmx服务

客户端采用如下代码连接本机的 Sentinel 服务端，并创建 MBean 的 ObjectName 对象：

```
JMXServiceURL url = new JMXServiceURL("service:jmx:rmi:///jndi/rmi://localhost:29096/jmxrmi");

JMXConnector jmxc = JMXConnectorFactory.connect(url, null);
MBeanServerConnection mbsc = jmxc.getMBeanServerConnection();

//ObjectName 的名称与前面注册时候的保持一致
ObjectName mbeanName = new ObjectName("jmxBean:name=rdsCenter");
```

8.2 获取进程标识

执行如下代码获取当前 TongRDS 的进程标识：

```
// 实例标识名
String id = (String) mbsc.getAttribute(mbeanName, "Id");
```

8.3 获取软件版本号

执行如下代码获取当前 TongRDS 服务的版本信息：

```
// 版本号
String version = (String) mbsc.getAttribute(mbeanName, "Version");
```

Version 为字符串形式返回的版本信息。

8.4 获取license使用量

执行如下代码获取 license 使用量：

```
// 已经被使用掉的 license 总内存
Long usedLicense = (Long) mbsc.getAttribute(mbeanName, "UsedLicense");
```

8.5 获取license总授权量

执行如下代码获取 license 总授权量：

```
// 当前授权的总 license 内存可用量
Long totalLicense = (Long) mbsc.getAttribute(mbeanName, "TotalLicense");
```

8.6 获取license授权的过期时间

执行如下代码获取 license 授权的过期时间：

```
// 当前授权的过期时间
Long licenseExpired = (Long) mbsc.getAttribute(mbeanName, "LicenseExpired");
```

8.7 获取JVM已分配的内存总量

该值表示 JVM 已经从操作系统中申请的内存空间。执行如下代码获取 JVM 已分配的内存：

```
// jvm 已经分配的内存
Long jvmAllocated = (Long) mbsc.getAttribute(mbeanName, "JvmAllocated");
```

8.8 获取JVM空闲的内存量

该值表示 JVM 已经从操作系统中申请的内存空间中尚未使用的部分。执行如下代码获取 JVM 已分配的内存：

```
// jvm 已经分配但空闲的内存
Long jvmFree = (Long) mbsc.getAttribute(mbeanName, "JvmFree");
```

注：JVM 已使用的内存总量可使用上述 2 项的值进行减法计算获得。

8.9 获取JVM可分配的内存总量

该值表示当前进程可分配的内存总量（可由启动参数-Xmx 指定）执行如下代码获取 JVM 已分配的内存：

```
// jvm 可分配的最大内存
Long jvmMax = (Long) mbsc.getAttribute(mbeanName, "JvmMax");
```

8.10 获取当前连接的客户端数量

执行如下代码获取 TongRDS 当前正在连接的客户端总数：

```
// 当前接入的 client 连接数量
Long clientCurrentConnections = (Long) mbsc.getAttribute(mbeanName, "ClientCurrentConnections");
```

8.11 Jmx客户端例程

以下为 jmx 演示例程：

```
public void check(Statistics statistics/* pojo */) {
    if (jmxUrl != null && !jmxUrl.startsWith("service:jmx")) {
        jmxUrl = "service:jmx:rmi:///jndi/rmi://" + jmxUrl + "/jmxrmi";
    }

    JMXConnector jmxnc = null;
    try {
        JMXServiceURL url = new JMXServiceURL(jmxUrl);

        jmxnc = JMXConnectorFactory.connect(url, null);

        // 程序能运行到这里说明已经连接上 jmx-server 了
        statistics.setAlive(true);

        MBeanServerConnection mbsc = jmxnc.getMBeanServerConnection();

        //ObjectName 的名称与前面注册时候的保持一致
        ObjectName mbeanName = new ObjectName("jmxBean:name=rdsCenter");

        // 实例标识名
        statistics.setIdentifier((String) mbsc.getAttribute(mbeanName, "Id"));

        // 版本号
        statistics.setVersion((String) mbsc.getAttribute(mbeanName, "Version"));
    } catch (Exception e) {
        // 连接失败
    }
}
```



```

// 已经被使用掉的 license 总内存
statistics.setUsedLicense((Long) mbsc.getAttribute(mbeanName, "UsedLicense"));

// 当前授权的总 license 内存可用量
statistics.setTotalLicense((Long) mbsc.getAttribute(mbeanName, "TotalLicense"));

// 当前授权的过期时间
statistics.setLicenseExpired((Long) mbsc.getAttribute(mbeanName, "LicenseExpired"));

// jvm 已经分配的内存
statistics.setJvmAllocated((Long) mbsc.getAttribute(mbeanName, "JvmAllocated"));

// jvm 已经分配但空闲的内存
statistics.setJvmFree((Long) mbsc.getAttribute(mbeanName, "JvmFree"));

// jvm 可分配的最大内存
statistics.setJvmMax((Long) mbsc.getAttribute(mbeanName, "JvmMax"));

// 当前接入的 client 连接数量
statistics.setCurrentConnections((Long) mbsc.getAttribute(mbeanName, "ClientCurrentConnections
"));

    LOGGER.infoLog("CenterDeployer::check() {} read jmx-server '{}' ok.", name, jmxUrl);
} catch (AttributeNotFoundException anf) {
    if (LOGGER.isDebugEnabled()) {
        LOGGER.debugLog(anf, "CenterDeployer::check() {} read jmx-server '{}' failed: {}", name, jmx
Url, anf.getMessage());
    } else {
        LOGGER.errorLog("CenterDeployer::check() {} read jmx-server '{}' failed: {}", name, jmxUrl, a
nf.getMessage());
    }
} catch (IOException ioe) {
    this.statistics.setAlive(false);
    if (LOGGER.isDebugEnabled()) {

```

```

        LOGGER.debugLog(ioe, "CenterDeployer::check() {} connect to jmx-server '{}' failed: {}", name, jmxUrl, ioe);
    } else {
        LOGGER.infoLog("CenterDeployer::check() {} connect to jmx-server '{}' failed: {}", name, jmxUrl, ioe);
    }
} catch (Throwable t) {
    if (LOGGER.isDebugEnabled()) {
        LOGGER.debugLog(t, "CenterDeployer::check() {} jmx failed from '{}' failed: {}", name, jmxUrl, t);
    } else {
        LOGGER.infoLog(t, "CenterDeployer::check() {} jmx failed from '{}' failed: {}", name, jmxUrl, t);
    }
} finally {
    try {
        jmx.close();
    } catch (Throwable e) {
        // e.printStackTrace();
    }
}
if (LOGGER.isDebugEnabled()) {
    LOGGER.debugLog("CenterDeployer::check() {}'s statistics: {}", this.name, this.statistics);
}
}

```

附录A 客户端API演示程序

本程序样例实现了 Socket 连接、集群多节点异常切换等功能。实现了最常用的 set 和 get 接口。

```
import java.io.*;
import java.net.*;
import java.util.concurrent.atomic.AtomicLong;

/**
 * Clinet 演示
 */
public class ClientDemo {
    private final InetSocketAddress[] Addresses;

    private final String Password;

    private int CurrentAddr = 0;

    private Socket CurrentSocket = null;
    private Writer CurrentWriter = null;
    private BufferedReader CurrentReader = null;

    private AtomicLong SessionId = new AtomicLong(0);

    /**
     * @param passwd 如果 Server 端设置了需要密码认证，此处为密码，否则要必须为 null
     * @param addr 服务器的地址列表
     */
    public ClientDemo(String passwd, InetSocketAddress... addr) {
        if (addr == null || addr.length <= 0) {
            throw new IllegalArgumentException("Address is null");
        }

        Password = passwd;
    }
}
```

```

Addresses = addrs;

exchange();
}

private void exchange() {
    if (CurrentSocket != null) {
        try {
            CurrentSocket.close();
        } catch (Throwable e) {
            //e.printStackTrace();
        }
    }
    for (int i = 0; i < Addresses.length; i++) {
        try {
            CurrentSocket = new Socket();
            CurrentSocket.setTcpNodelay(true);
            CurrentSocket.setSoTimeout(1000);// 1 second
            CurrentSocket.connect(Addresses[CurrentAddr], 1000);
            CurrentWriter = new OutputStreamWriter(CurrentSocket.getOutputStream());
            if (Password != null) {
                CurrentWriter.write(Password);
                CurrentWriter.write("\n");
                CurrentWriter.flush();
            }
            CurrentReader = new BufferedReader(new InputStreamReader(CurrentSocket.getInputStream()));
            break;
        } catch (Exception e) {
            //e.printStackTrace();
        }
        CurrentAddr = (CurrentAddr + 1) % Addresses.length;
    }
}

```

```

/**
 * set 命令
 *
 * @param key
 * @param value
 * @return
 */
public synchronized boolean set(String key, String value) {
    String sess_id = Long.toString(SessionId.incrementAndGet());
    String resp = null;
    try {
        CurrentWriter.write(sess_id + " set 1 " + key + " " + value + "\n");
        CurrentWriter.flush();
        resp = CurrentReader.readLine();
    } catch (Exception e1) {
        exchange();
        try {
            CurrentWriter.write(sess_id + " set 1 " + key + " " + value + "\n");
            CurrentWriter.flush();
            resp = CurrentReader.readLine();
        } catch (Exception e) {
        }
    }

    return resp != null && resp.startsWith(sess_id + " ok");
}

/**
 * get 命令
 *
 * @param key
 * @return
 */
public synchronized String get(String key) {
    String sess_id = Long.toString(SessionId.incrementAndGet());

```

```
String resp = null;
try {
    CurrentWriter.write(sess_id + " get 1 " + key + "\n");
    CurrentWriter.flush();
    resp = CurrentReader.readLine();
} catch (Exception e1) {
    exchange();
    try {
        CurrentWriter.write(sess_id + " get 1 " + key + "\n");
        CurrentWriter.flush();
        resp = CurrentReader.readLine();
    } catch (Exception e) {
    }
}
String head = sess_id + " ok ";
if (resp != null && resp.startsWith(head)) {
    return resp.substring(head.length());
}
return null;
}

public static void main(String[] argv) {
    InetAddress addr1 = new InetAddress("110.50.1.55", 16200);
    InetAddress addr2 = new InetAddress("110.50.1.56", 16200);
    InetAddress addr3 = new InetAddress("110.50.1.57", 16200);

    ClientDemo client = new ClientDemo(null, addr1, addr2, addr3);

    boolean ret = client.set("key1", "value1");

    System.out.println(client.get("key1"));
}
}
```

附录B 术语说明

名称	说明
TongRDS (简称 RDS)	东方通分布式内存数据缓存中间件
中心节点	提供服务节点注册、运行模式管理功能的管理节点
服务节点	实际提供数据缓存服务的功能节点

